

TravelView 网站上线手册

目标：拿到一个能对外分享的公开链接，并且能收钱。全程约 2 小时，其中一大半是等 DNS 和填 Stripe 表单。

0. 部署到你的 OCI 服务器

和 TensuGo 同机，nginx 共用 443。代码是 output: 'standalone'，不绑任何托管商，所以将来想搬也只是换个地方 docker run。

先说清楚照片在哪，因为这件事最容易被误会：

东西	存在哪	会不会进 Git
原图	只在用户自己的电脑上	永远不会
发布用的 WebP（已剥 EXIF）	对象存储（OCI Object Storage）	不会
用户、订单、Story manifest	Postgres	不会
代码	Git 仓库	只有代码

Git 仓库里一张照片都没有，.gitignore 也挡着导出目录。照片走的是「桌面端 → 预签名地址 → 对象存储」这条线，连我们的服务器都不经过，更不可能进仓库。

如果哪天想图省事，Vercel + Neon 那条路也是通的（见附录 A），数据面完全一样。

1. 准备四样外部服务

顺序有讲究：**R2** 和 **Stripe** 要先拿到值，部署时才填得进去。

1.1 Postgres（就装在 OCI 那台）

TensuGo 那台上如果已经有 Postgres，直接建一个库和一个独立用户就行 —— 不要和 TensuGo 共用同一个数据库用户，出事时好隔离。

```
sudo -u postgres psql <<'SQL'
CREATE USER travelview WITH PASSWORD '换成强密码';
CREATE DATABASE travelview OWNER travelview;
SQL

cd web
psql "postgres://travelview: 密码 @127.0.0.1:5432/travelview" -f db/schema.sql
```

连接串就是 DATABASE_URL。走本机回环不用 sslmode=require。

1.2 对象存储 (OCI Object Storage)

代码用的是 S3 兼容接口，换供应商只改环境变量，不动代码。在 OCI 上就用它自家的 Object Storage：和服务器同区域，走内网不计出网费。

1. OCI 控制台 → Storage → Buckets → 建桶 travelview
2. 记下 **Namespace**(在桶详情页),接入点长这样:https://<namespace>.compat.objectstorage.<region>.oraclecloud.com
3. 用户菜单 → My profile → **Customer secret keys** → 生成一对，这就是 S3 兼容用的 Access Key / Secret Key（不是 API 签名密钥，别拿错了）
4. 图片要能公开读，两种做法二选一：
 - 桶的 Visibility 设为 **Public**,NEXT_PUBLIC_MEDIA_BASE 直接指到 https://<namespace>.objectstorage.<region>.oraclecloud.com
 - 或者桶保持私有，用 nginx 在 media. 你的域名 上反代它（更好看，也好换）

对应的环境变量：

```
S3_ENDPOINT=https://<namespace>.compat.objectstorage.<region>.oraclecloud.com
S3_REGION=us-ashburn-1           # 换成你的区域
S3_FORCE_PATH_STYLE=true         # OCI 必须是 true
S3_ACCESS_KEY_ID=...
S3_SECRET_ACCESS_KEY=...
S3_BUCKET=travelview
```

不用配 **CORS**。桌面端是原生 HTTP 客户端直传，不走浏览器，没有跨域这回事。以后要是做网页版上传，再回来加。

出网费要盯着。OCI 每月有免费出网额度，超了按量收。图片型产品的成本几乎全在流量上——万一某篇 Story 火了，换成 Cloudflare R2（零出网费）只需要改上面那几行。

1.3 Stripe

1. 建两个 Price：单篇发布（一次性）和年费（订阅，可以先不做）
2. 记下两个 price_... ID
3. Webhook 等网站部署完拿到域名再配（见 3.3）——顺序反了会配错地址

1.4 域名

主域名 A 记录指到 OCI 那台的公网 IP；media. 子域也指过去（由 nginx 反代对象存储），或者直接 CNAME 到桶的公开地址。

证书用 certbot，和 TensuGo 那套一样：

```
sudo certbot --nginx -d 你的域名 -d media. 你的域名
```

2. 部署

2.1 写 .env.production

在服务器上，放在仓库外面（例如 /etc/travelview/env），chmod 600。清单见 web/.env.example：

变量	值
DATABASE_URL	postgres://travelview: 密码@127.0.0.1:5432/travelview
AUTH_SECRET	openssl rand -base64 32 现生成
AUTH_URL	https://你的域名
STRIPE_SECRET_KEY	sk_live_... (先用 sk_test_... 跑通)
STRIPE_WEBHOOK_SECRET	3.3 拿到后再回来填
STRIPE_PRICE_ONETIME	price_...
S3_ENDPOINT / S3_REGION /	见 1.2
S3_FORCE_PATH_STYLE	
S3_ACCESS_KEY_ID / S3_SECRET_ACCESS_KEY /	见 1.2
S3_BUCKET	
NEXT_PUBLIC_MEDIA_BASE	https://media. 你的域名
NEXT_PUBLIC_SITE_URL	https://你的域名
NEXT_PUBLIC_MAP_TILES	暂时留空, 见 4.2

推之前在本地自检一遍, 省一次白跑的构建:

```
cd web
npm install
set -a; source .env.local; set +a
npm run preflight # 缺哪个变量会直接列出来
```

2.2 构建并跑起来

```
cd web
docker build \
  --build-arg NEXT_PUBLIC_SITE_URL=https://你的域名 \
  --build-arg NEXT_PUBLIC_MEDIA_BASE=https://media. 你的域名 \
  -t travelview-web .
```

```
docker run -d --name travelview-web \
  --env-file .env.production \
  -p 3001:3000 --restart unless-stopped \
  travelview-web
```

NEXT_PUBLIC_* 必须在 **build** 时给 (它们被编译进前端代码), 其余密钥在 **run** 时给。搞反了的表现是: 页面上的域名全是空的。

2.3 nginx

前面用 nginx 反代, 和 TensuGo 共用 443:

```
location / {
    proxy_pass http://127.0.0.1:3001;
```

```

    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

```

Stripe webhook 需要原始请求体验签，所以反代不能改写 **body**，也不要在这个 location 上加任何缓冲/压缩的花样。

桶保持私有的话，media. 你的域名 这个 server 块再加一段反代到对象存储，并且给图片配长缓存 —— 图片一旦发布就不再变：

```

location / {
    proxy_pass https://<namespace>.objectstorage.<region>.oraclecloud.com/n/<ns>/b/travelview;
    proxy_hide_header Set-Cookie;
    add_header Cache-Control "public, max-age=31536000, immutable";
}

```

2.4 升级怎么做

```

git pull && cd web
docker build --build-arg NEXT_PUBLIC_SITE_URL=https://你的域名 \
  --build-arg NEXT_PUBLIC_MEDIA_BASE=https://media.你的域名 \
  -t travelview-web:new .
docker stop travelview-web && docker rm travelview-web
docker tag travelview-web:new travelview-web
docker run -d --name travelview-web --env-file /etc/travelview/env \
  -p 3001:3000 --restart unless-stopped travelview-web

```

回滚：旧镜像别急着删 (docker images)，出问题把上一个 tag 重新 docker run 起来就行。数据库改动要能兼容旧代码，否则回滚会失败 —— 目前 db/schema.sql 只增不改，满足这一点。

3. 上线后必须做的三件事

3.1 冒烟测试（按这个顺序，一步不对就停下）

1. 打开 https://你的域名 —— 落地页出来
2. 注册一个账号 —— 能登录
3. /pricing 点付款，用测试卡 4242 4242 4242 4242，任意未来日期
4. 回到 /account —— 额度加上了（没加就是 webhook 没通，看 3.3）
5. /account 生成一个发布令牌
6. 桌面端「生成旅行回顾」→ 导出 → 发布，填域名和令牌
7. 打开返回的公开链接 —— 照片、地图、路线都在

3.2 把公开链接粘到 Facebook 看一眼

卡片必须有大图和标题。不对就用 Facebook 分享调试器点 “Scrape Again” —— 它缓存得很凶，改完 OG 不重新抓是看不到的。

3.3 Stripe Webhook

权益只在 **webhook** 里发放，所以这一步不通，用户付了钱也拿不到额度。

- 1. Stripe → Developers → Webhooks → Add endpoint
- 2. 地址 `https://你的域名/api/stripe/webhook`
- 3. 事件选 `checkout.session.completed` 和 `customer.subscription.updated`、`customer.subscription.deleted`
- 4. 把 `whsec_...` 填回 `STRIPE_WEBHOOK_SECRET`，重新部署
- 5. 在 Stripe 后台点 “Send test webhook” 确认返回 200

本地调试用：

```
stripe listen --forward-to localhost:3000/api/stripe/webhook
```

4. 上线前的两个欠账

这两件事不影响跑通，但影响能不能长期跑。

4.1 数据库备份

自托管没有人替你做这件事。至少来一条定时任务，并且验证过能恢复 —— 没恢复过的备份不算备份：

```
# /etc/cron.daily/travelview-backup
pg_dump "postgres://travelview: 密码 @127.0.0.1/travelview" \
| gzip > /var/backups/tv-$(date +%F).sql.gz
find /var/backups -name 'tv-*.sql.gz' -mtime +30 -delete
```

对象存储里的图片同样要有一份 —— 用户发布的 Story 丢了，是没法从原图重建的（原图在他自己电脑上，我们连有哪些都不知道）。

4.2 地图瓦片

现在指着 `tile.openstreetmap.org`。那是给开发和小流量用的公益服务，正式产品流量会违反它的使用政策，而且把访客的浏览行踪送给了第三方。上线前换成自托管的 Protomaps，见 `Design/map-tiles.md`。

5. 出问题时先看哪里

现象	多半是
构建失败，找不到 <code>package.json</code>	Vercel 的 Root Directory 没设成 web
页面能开，登录报错	AUTH_SECRET 或 AUTH_URL 没填 / 填错
付了钱额度不加	webhook 没配、密钥不对，或反代改写了 body
Story 页面文字在、图片全裂	NEXT_PUBLIC_MEDIA_BASE 不对，或桶没开公开访问 / nginx 没反代

现象	多半是
上传返回 403 SignatureDoesNotMatch	OCI 忘了 S3_FORCE_PATH_STYLE=true, 或用错了密钥 (要 Customer secret key)
发布时返回 401	令牌被吊销了, 去 /account 重新生成
发布时返回 402	额度用完了 —— 这是正常行为, 不是故障
FB 卡片没图	用分享调试器 Scrape Again; OG 图必须是 PNG, 不能是 WebP

6. 一页纸清单

- [] OCI 上建库建用户 + psql -f db/schema.sql
- [] 建对象存储桶 + Customer secret key + 图片能公开访问
- [] Stripe 建 Price, 记下 price_id
- [] 域名解析 + certbot 证书
- [] 写 /etc/travelview/env (npm run preflight 自检)
- [] docker build + run + nginx 反代
- [] 打开落地页
- [] 注册 -> 付款 -> 额度到账
- [] 生成发布令牌 -> 桌面端发布 -> 打开公开链接
- [] 配 Stripe webhook -> 重新部署 -> 发测试事件
- [] FB 分享调试器看卡片
- [] 换掉地图瓦片
- [] 数据库备份

附录 A. 改用 Vercel + Neon (可选)

A.1 连仓库

Vercel → New Project → 选这个 GitHub 仓库 → **Root Directory** 一定要设成 **web** (仓库根目录是 Flutter 项目, 不设这一项会构建失败)。

A.2 环境变量

在 Vercel 的 Settings → Environment Variables 里填。清单和说明见 web/.env.example, 一个都不能少:

变量	值
DATABASE_URL	Neon 的连接串 (neon.tech 建库后复制)
AUTH_SECRET	openssl rand -base64 32 现生成
AUTH_URL	https://你的域名
STRIPE_SECRET_KEY	sk_live_... (先用 sk_test_... 跑通)
STRIPE_WEBHOOK_SECRET	3.3 拿到后再回来填
STRIPE_PRICE_ONETIME	price_...

变量	值
S3_ENDPOINT / S3_ACCESS_KEY_ID / S3_SECRET_ACCESS_KEY / S3_BUCKET	见 1.2; 用 R2 的话接入点是 https://<account_id>.r2.cloudflarestorage.com
NEXT_PUBLIC_MEDIA_BASE	https://media. 你的域名
NEXT_PUBLIC_SITE_URL	https://你的域名
NEXT_PUBLIC_MAP_TILES	暂时留空, 见 4.2

NEXT_PUBLIC_ 开头的会编译进前端代码, 改了必须重新部署才生效。

A.3 部署

推到 main 就会自动构建。本地先自检一遍, 省一次失败的构建:

```
cd web
npm install
npm run preflight # 缺哪个变量会直接列出来
npm run build
```